

Exercise 1: Configuring a Basic Spring Application

Scenario: Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. **Set Up a Spring Project:**
 - Create a Maven project named **LibraryManagement**.
 - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
 - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
 - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
 - Create a package **com.library.service** and add a class **BookService**.
 - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
 - Create a main class to load the Spring context and test the configuration.

Solution:

1. **Set Up a Spring Project:**
 1. Maven project : “ maven-archetype-quickstart “ (Version 1.5)

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

☒ run archetype generation interactively

Properties available from archetype:

Name	Value
javaCompilerVersion	17
junitVersion	5.11.0

Advanced

< Back Next > Finish Cancel

2. pom.xml file – add dependency

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.14</version>
</dependency>
```

2. **Configure the Application Context:**

1. applicationContext.xml – created resources folder in the src/main/java

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="bookRepository"
    class="com.library.repository.BookRepository"/>
  <bean id="bookService"
    class="com.library.service.BookService">
    <property name="bookRepository"
      ref="bookRepository"/>
  </bean>
</beans>
```

3. **Define Service and Repository Classes:**

1. BookService.java – created and saved in the respected package

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
  private BookRepository bookRepository;
  public void setBookRepository(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
  }
  public void showBooks() {
    System.out.println("Book Service Called...");
    bookRepository.displayBooks();
  }
}
```

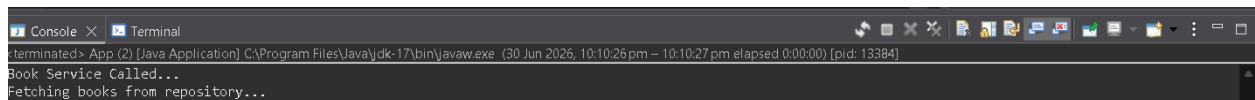
2. BookRepository.java – created and saved in the respective package

```
package com.library.repository;
public class BookRepository {
  public void displayBooks() {
    System.out.println("Fetching books from repository...");
  }
}
```

4. Run the Application:

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class App {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml"
            );
        BookService service =
            context.getBean(
                "bookService",
                BookService.class
            );
        service.showBooks();
    }
}
```

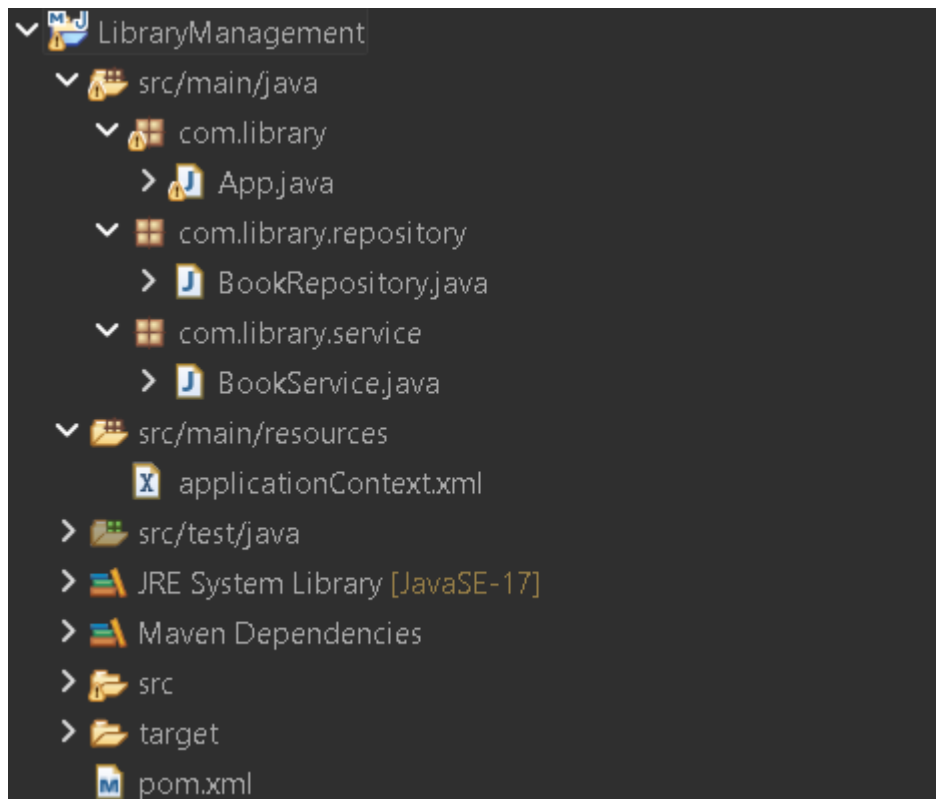
Output:



The screenshot shows a terminal window with the following output:

```
terminated> App (2) [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (30 Jun 2026, 10:10:26pm - 10:10:27pm elapsed 0:00:00) [pid: 13384]
Book Service Called...
Fetching books from repository...
```

Project :



Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the `BookService` and `BookRepository` classes using Spring's IoC and DI.

Steps:

1. **Modify the XML Configuration:**
 - Update `applicationContext.xml` to wire `BookRepository` into `BookService`.
2. **Update the BookService Class:**
 - Ensure that `BookService` class has a setter method for `BookRepository`.
3. **Test the Configuration:**
 - Run the `LibraryManagementApplication` main class to verify the dependency injection.

Solution:

1. **Modify the XML Configuration**

Added in the `bookService` bean tags

```
<property name="bookRepository" ref="bookRepository"/>
```

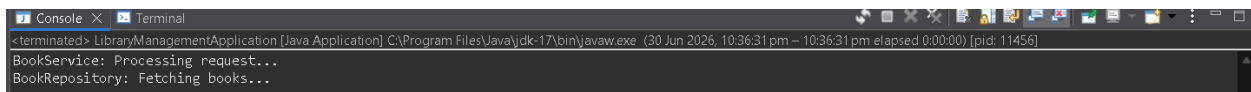
2. **Update the BookService Class**

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
    private BookRepository bookRepository;
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void issueBook() {
        System.out.println("BookService: Processing request...");
        bookRepository.displayBooks();
    }
}
```

3. Test the Configuration

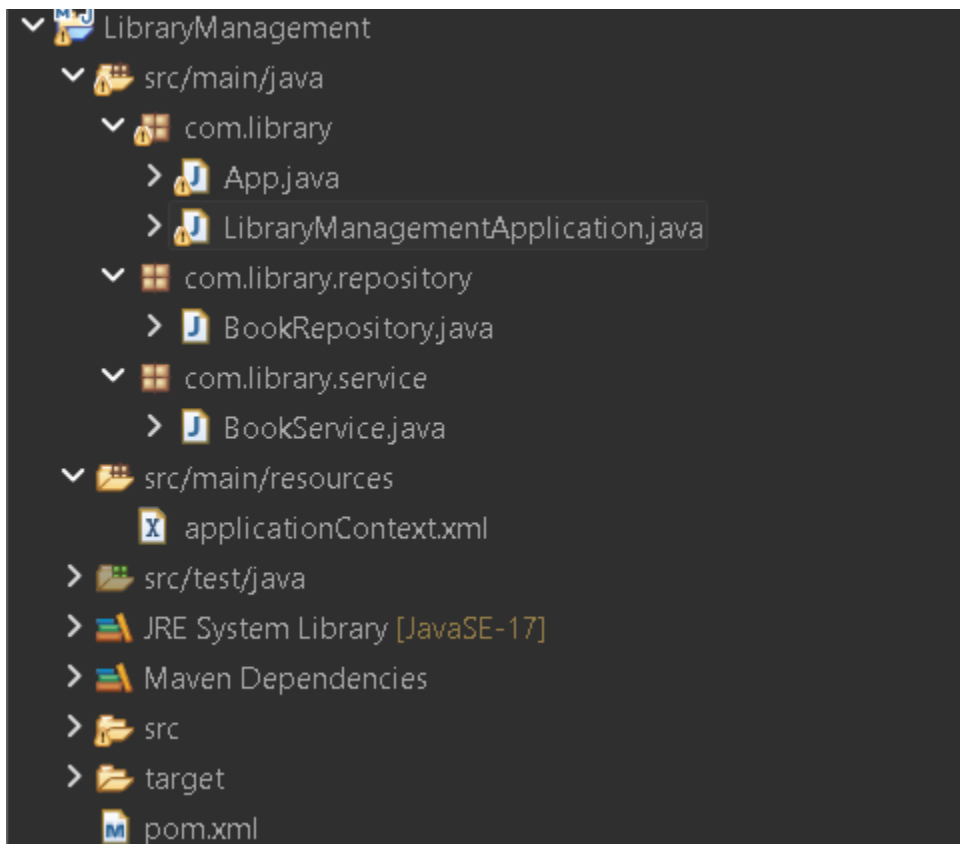
```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml"
            );
        BookService service =
            context.getBean(
                "bookService",
                BookService.class
            );
        service.issueBook();
    }
}
```

Output:



A screenshot of a terminal window. The title bar shows 'Console' and 'Terminal'. The output text is: '<terminated> LibraryManagementApplication [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe. (30 Jun 2026, 10:36:31 pm - 10:36:31 pm elapsed 0:00:00) [pid: 11456]'. Below this, two lines of output are visible: 'BookService: Processing request...' and 'BookRepository: Fetching books...'. The terminal has a dark background with light-colored text.

Project:



Exercise 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. **Create a New Maven Project:**
 - Create a new Maven project named **LibraryManagement**.
2. **Add Spring Dependencies in pom.xml:**
 - Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.
3. **Configure Maven Plugins:**
 - Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

Solution:

1. Project has been created check the above exercises
2. Added the Dependencies

```
<dependencies>
  <!-- Spring Context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version>
  </dependency>
  <!-- Spring AOP -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>${spring.version}</version>
  </dependency>
  <!-- Spring Web MVC -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>${spring.version}</version>
  </dependency>
</dependencies>
```

3. Added Pulgin

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>
        maven-compiler-plugin
    </artifactId>
    <version>3.13.0</version>
    <configuration>
        <source>1.8</source>
        <target>1.8</target>
    </configuration>
</plugin>
```

Compilation: mvn clean compile

```
D:\PROJECTS\SPRING PROJECTS\LibraryManagement>mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ LibraryManagement ---
[INFO] Deleting D:\PROJECTS\SPRING PROJECTS\LibraryManagement\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ LibraryManagement ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ LibraryManagement ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 4 source files with javac [debug target 1.8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 1.352 s
[INFO] Finished at: 2026-06-30T22:48:09+05:30
[INFO] -----
D:\PROJECTS\SPRING PROJECTS\LibraryManagement>
```